

EDL → Akamai Client List Connector

Test-Account Setup & Run Guide

Step-by-step: configure the connector so each managed customer's Akamai Client List ("Singtel_IOC_List") is populated from the central EDL feed, including the firewall whitelist.

Version 1.0 — 13 July 2026 | Standard-library Python connector

1. What this does, and one hard rule

The connector reads the EDL IOC Manager's `Block_IoC_IPs.txt` and pushes the IP/CIDR entries into an **existing** Akamai **Client List**, then activates it, so App & API Protector (AAP/ASM) enforces them. It is outbound-only, standard-library only, and multi-tenant (each customer account is one tenant).

IMPORTANT: This tool **never creates** a Network List or Client List. It only updates a list that already exists. If the target list doesn't exist, it stops with a clear error rather than creating anything — so **Singtel_IOC_List** must be created in each customer's Akamai account first (Step 4).

Tenants are folders. The connector reads a **Client/** folder next to the script; each subfolder is one managed customer (folder name = tenant name). Each customer folder holds that customer's own `.edgerc` credential (with their **account_switch_key** on the last line); a folder without one falls back to a default `.edgerc` beside **Client/**.

The setup, end to end: whitelist one hostname on the firewall (Step 2) → generate an Akamai API credential per account (Step 3) → create **Singtel_IOC_List** and wire its DENY into policy (Step 4) → place the files & per-tenant folders on the host (Step 5) → edit the config (Step 6) → run offline → dry-run → push (Step 7) → schedule it (Step 8).

2. Firewall — the exact hostname to whitelist

The script makes an **outbound** HTTPS connection from the host to Akamai. There is nothing inbound to open. The destination is the per-credential API hostname — the **exact value on the host line of your** `.edgerc`, which looks like:

WHITELIST THIS: `akab-XXXXXXXXXXXXXXXX-XXXXXXXXXXXXXXXX.luna.akamaiapis.net` — outbound **TCP 443** only. Copy the literal value from your `.edgerc`. If your firewall's FQDN objects struggle with the many short-TTL A records, use the wildcard `*.luna.akamaiapis.net`.

Field	Value
Direction	Outbound / egress (host → Akamai). No inbound rule.
Destination	The host from <code>.edgerc</code> , e.g. <code>akab-....luna.akamaiapis.net</code> (fallback: <code>*.luna.akamaiapis.net</code>)
Port	TCP 443 (HTTPS) only
Match by	FQDN, not IP — Akamai edge IPs rotate and are not published; an IP allowlist will break
Also needed	DNS resolution for that FQDN from the host (FQDN rules depend on it)

NOTE: If your egress passes through a **TLS-inspection proxy**, it re-signs Akamai's certificate with a corporate CA. The connector handles this — see Step 7 (`akamai_ca_bundle`). If egress uses an **explicit proxy**, set an `HTTPS_PROXY` environment variable on the task.

3. Generate the Akamai API credential (test account)

Do this while logged in to Akamai Control Center as an admin, **switched into the TEST account** (an API client is anchored to whatever account you are in when you create it).

#	Step
3.1	Use the account selector to switch into the test account .
3.2	Open  → ACCOUNT ADMIN → Identity & access , then Create API client .
3.3	Choose Advanced (not Quick) so you can scope the client.
3.4	Group: pick the group that contains the Client List you will manage (narrowest that still contains it).
3.5	APIs: search "client list" , add that service, set it to READ-WRITE (needed to add items and activate). Leave everything else off. Do NOT pick "Network Lists" — that is a different product.

#	Step
3.6	Leave “manage multiple accounts” unticked for a test-only client.
3.7	Click Create API client , then Download the credentials. The <code>client_secret</code> is shown only once — save it now.

You end up with four values. Save them as a file named exactly `.edgerc` with a `[default]` section, and add the customer’s **account switch key on the last line** (this is how the connector targets the right account — omit it only for the account the credential is natively scoped to):

```
[default]
host = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx.luna.akamaiapis.net
client_token = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx
client_secret = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx=
access_token = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx
account_switch_key = 1-XXXXXX:1-XXX
```

NOTE: If you don’t see **Create API client**, you lack API-provisioning rights — a Primary Admin must grant them or create the client for you. A client can only be granted APIs already enabled on the account, so confirm **Client Lists** is enabled on the test account.

4. Create Singtel_IOC_List & wire the DENY (one-time, per account)

Because the connector never creates lists, create the target list in **each managed account** first:

#	Step
4.1	In the account, create a Client List of type IP named exactly Singtel_IOC_List (the connector matches on the exact name). It can start empty.
4.2	In that account's App & API Protector security policy, reference the list in the IP/Geo controls with a DENY action. This is what actually blocks traffic — the connector only manages the list's membership, not the policy.
4.3	The connector finds the list by name, so you don't need to copy its ID. If two lists ever share the name in one account, the connector stops and asks you to disambiguate (it will not guess).

IMPORTANT: Enforcement is only real once the list is **referenced in the policy with DENY**. A list the policy doesn't reference blocks nothing, even when the connector fills it correctly.

5. Place the files & per-tenant folders on the host

#	Step
5.1	Unzip the connector to a folder outside any web root , e.g. C:\Tools\edl-akamai-connector, so its source and credentials are never web-served.
5.2	Create a Client folder next to connector.py, and one subfolder per managed customer. Put each customer's .edgerc (Step 3) inside their subfolder. A customer without their own file uses the default .edgerc placed next to the Client folder:
5.3	Lock every .edgerc ACL: remove inheritance; grant Read only to Administrators, SYSTEM, and the task's run-as account. Each holds a live API secret.
5.4	Make sure the host has Python 3.9+ installed and can read the EDL data folder (Block_IOC_IPs.txt).

```
edl-akamai-connector\  
connector.py  
config.json  
.edgerc                &lt;- default credential (fallback)  
Client\  
  CustomerA\.edgerc    &lt;- CustomerA's own credential (switch key on last line)  
  CustomerB\  
  CustomerC\.edgerc    &lt;- no .edgerc -&gt; uses the default above
```

TIP: For your first test, one subfolder is enough: Client\TestAccount\.edgerc containing the test account's credential. Add more customers later by adding more subfolders — no code or config change. Remove or rename a folder to opt that customer out.

6. Configure config.json

Copy config.example.json to config.json. It now holds only **global** settings — tenants and their switch keys come from the client/ folders and their .edgerc files, not from here.

```
{  
  "central_ioc_source": "C:\\inetpub\\edl-data\\Block_IOC_IPs.txt",  
  "client_list_name": "Singtel_IOC_List",  
  "activation_network": "STAGING",  
  "safety_shrink_pct": 30,  
  "akamai_verify_ssl": true  
}
```

Field	Meaning
central_ioc_source	Path to Block_IOC_IPs.txt — the one feed every tenant receives. For the very first push you may point this at a small test .txt (a few IPs) so the result is easy to eyeball.

Field	Meaning
client_list_name	The exact list name populated in every account. Defaults to Singtel_IOC_List; resolved to its ID at run time, never created.
activation_network	STAGING first; switch to PRODUCTION only after you confirm STAGING.
safety_shrink_pct	Abort (don't push) if the new set would delete more than this % of the live list — guards against a broken source wiping the list.

NOTE: The **account_switch_key** is not in this file — it lives on the last line of each customer's .edgerc (Step 3). A customer whose credential has no switch key is pushed to the account that credential is natively scoped to.

7. Run it — offline, then dry-run, then push

From the connector folder, in order. Each step is safe up to the last one.

7.1 Offline (no network) — proves parsing

```
python connector.py --config config.json --offline
```

Reads and parses the IP file, prints the counts. Makes no Akamai calls. Confirms the file path and that your IPs parse.

7.2 Dry-run (read-only Akamai call) — proves credential + firewall + list

```
python connector.py --config config.json --dry-run
```

Makes one **read-only** call per tenant folder: it resolves **Singtel_IOC_List** to its ID in that account, reads the current entries, and prints the exact add/remove diff — but writes nothing. If this succeeds, your firewall whitelist, the .edgerc (and its switch key), and the list name are all correct. A clean line looks like:

```
[--] TestAccount      dry-run      desired=3105 current=0 (+3105/-0)  no writes
```

7.3 Push (writes + activates on STAGING)

```
python connector.py --config config.json
```

Pushes the add/remove delta into the list and activates it on **STAGING**. Then verify in Control Center that **Singtel_IOC_List** shows the entries and the staging activation succeeded, and test that a listed IP is blocked. Only then set `activation_network` to `PRODUCTION` and run again.

NOTE: A non-zero exit code means a tenant aborted or errored — point your scheduler's alerting at it. The connector isolates tenants, so one failure does not stop the others.

7.4 If the dry-run fails with a certificate error

If you see `CERTIFICATE_VERIFY_FAILED`, a TLS-inspection proxy is re-signing the connection. The message tells you the fix. Preferred: set `akamai_ca_bundle` to your corporate root CA PEM (keeps verification on). Last resort: bypass with `--insecure` (or `"akamai_verify_ssl": false`) — it prints a security warning because, with verification off, an interceptor could tamper with the responses this tool acts on.

8. Schedule it (Windows Task Scheduler)

Nothing is configured in IIS Manager — the connector is a standalone script, not an IIS site. Create a scheduled task:

Tab	Setting
General	Run as a dedicated service account; "Run whether user is logged on or not." The account needs Read on the EDL data folder and the .edgerc, and "Log on as a batch job" rights.
Trigger	Your cadence (e.g. hourly) — how fresh you want Akamai to be.
Action	Program = <code>python.exe</code> ; Arguments = <code>connector.py --config config.json</code> ; Start in = the connector folder.
Settings	"If the task fails, restart." Watch the task's Last Run Result (the connector's exit code) for alerting.

TIP: Logs: every run writes `logs\connector.log` (rotates daily, keeps 10 days automatically). Each line is tagged with the client name, so `grep "[CustomerA]"` gives one customer's full history. The task's exit code is still your primary alert signal; the log is the durable per-client audit trail.

9. Troubleshooting

Symptom	Cause → fix
"No existing IP Client List named ..."	The list doesn't exist (the tool never creates one) → create it in Control Center (Step 4), matching the name exactly.
"Multiple Client Lists named ..."	Two lists share the name → set an explicit <code>client_list_id</code> in the tenant to disambiguate.
CERTIFICATE_VERIFY_FAILED	Corporate TLS-inspection proxy → set <code>akamai_ca_bundle</code> (recommended) or <code>--insecure</code> (last resort). See 7.4.
Connection timed out / cannot resolve host	Firewall egress not open, or DNS → whitelist outbound 443 to the <code>.edgerc</code> host / <code>*.luna.akamaiapis.net</code> (Step 2); confirm DNS.
HTTP 401 "not provisioned to manage multiple accounts"	The tenant's <code>.edgerc</code> has an account switch key but that API client can't switch accounts → remove the <code>account_key</code> line if the credential is already for that account, or have an admin provision the client for multi-account.
HTTP 403	The API client lacks the Client List READ-WRITE grant, or the group doesn't contain the list → fix the client's permissions/group (Step 3).
Fetch succeeds but nothing is blocked	The list isn't referenced in the AAP policy with DENY → Step 4.2.
accountSwitchKey error	Account switching doesn't work with a service-account API client → use a normal user-based client (only relevant once you go multi-account).

This guide covers the test-account path. For the architecture, security model, and the full multi-tenant/subscription configuration, see the connector's README and the project's other documentation.